AI English Pronunciation Assessment

System Architecture & Design Review

AUTHOR
Dodla Prasanna Kumar Reddy
GITHUB
github.com/PrasannaKumar260904/livo-pronunciation-checker
LIVE DEMO
livo-pronunciation-checker-blue.vercel.app

Project Overview

AI English Pronunciation Assessment is a full-stack web application that evaluates a 30–45 second English speech recording. The user uploads audio through a Next.js frontend; a FastAPI backend validates the file, transcribes it with Faster-Whisper, and runs a deterministic scoring engine that analyzes speaking rate, pauses, and segment timing to produce a pronunciation score and segment-level coaching highlights. An optional OpenAI (GPT-4o-mini) pass adds natural-language coaching feedback, with a graceful fallback when AI feedback is unavailable.

High-Level Architecture

Request Lifecycle

One deterministic pipeline: the browser captures audio, FastAPI validates and transcribes it, a rule-based engine scores pronunciation and builds highlights, and an optional GPT-4o-mini pass adds coaching notes before the JSON renders on the dashboard.

Design principle

The scoring path has no external dependency — OpenAI feedback is additive only, so the app degrades gracefully if the AI provider is unavailable.

Component Responsibilities

Component	Responsibility	Examples
Frontend	Collects the audio sample, calls the backend REST API, and renders results.	Next.js App Router, TypeScript, Tailwind CSS
Backend	Orchestrates the assessment pipeline and exposes a JSON REST API.	FastAPI, Python 3.11, Pydantic
Upload Service	Receives the uploaded audio file over HTTPS.	FastAPI file upload endpoint
Validation Service	Rejects requests that fail duration, size, or format checks before processing.	Duration / file-size / audio-format validation, Mutagen
Transcription Service	Converts speech to text with word/segment timestamps.	Faster-Whisper (CPU inference)
Pronunciation Analysis	Computes a deterministic score from speaking rate, pauses, and transcript stats.	Rate, pause count, transcript length, avg. segment duration
Pronunciation Highlights	Generates segment-level coaching observations from timing and density signals.	Heuristic segment scoring, not phoneme-level
Feedback Service	Optionally enriches results with natural-language coaching; degrades gracefully.	OpenAI Chat Completions, GPT-4o-mini, fallback logic

■ Models & APIs Used — Rationale

Technology	Role	Why It Was Chosen
Faster-Whisper	Speech-to-text transcription	CTranslate2-based re-implementation of Whisper that runs efficiently on CPU with low memory footprint — avoids the GPU dependency of heavier ASR stacks while keeping accuracy high enough for short clips.
FastAPI	Backend API framework	Async-first, typed with Pydantic, and fast to validate/serialize audio-upload payloads; lighter and quicker to iterate on than Django for a single-purpose scoring service.
Next.js (App Router)	Frontend framework	Built-in routing, server components, and first-class TypeScript support give a fast, production-ready UI without a separate bundler/config layer.
OpenAI Chat Completions (GPT-4o-mini)	Optional coaching feedback	Low-cost, low-latency model that is more than sufficient for short, templated coaching text; kept strictly optional so the core pipeline never depends on it.

Pronunciation Scoring

The score is computed deterministically from measurable signals — no black-box phoneme model is involved:

- Speaking rate (words per minute)
- Pause count and pause duration
- Transcript length vs. expected range
- Average segment duration

The system intentionally avoids unsupported phoneme-level accuracy claims; it scores fluency-adjacent signals it can measure reliably.

Pronunciation Highlights

Segment-level coaching observations are generated heuristically from:

- Speaking rate per segment
- Pauses within/around a segment
- Segment timing and length
- Local transcript density

These are coaching observations, not phoneme-level pronunciation judgments.

■ DPDP Compliance

Aspect	Implementation
Consent	Audio is submitted voluntarily by the user for the sole purpose of assessment.
Storage	Uploaded audio is written to temporary storage only, never a persistent data store.
Retention	No long-term retention — files exist only for the duration of processing.
Deletion	Audio files are deleted automatically and immediately after the assessment completes.
Data residency	Frontend is hosted on Vercel; backend is hosted on Render.
Personal data handling	No permanent user accounts, no persistent audio storage, no profile data collected.

✓ Privacy by design

Because there is no persistent storage layer, the application has no standing repository of personal audio to protect, secure, or eventually purge — deletion happens as a natural side effect of the request lifecycle rather than a scheduled job.

■ Trade-offs

- Chose Faster-Whisper for efficient CPU inference over GPU-bound alternatives.
- Used deterministic heuristics instead of full phoneme alignment — simpler, explainable, and dependency-free.
- AI coaching feedback is optional by design, not a hard dependency of the scoring path.
- Graceful fallback when the AI provider is unavailable improves overall reliability.
- Free-tier deployment (Vercel + Render) increases cold-start latency but minimizes cost.

■ Future Improvements

- Word-level pronunciation assessment
- Phoneme alignment
- Forced alignment
- WhisperX integration
- Streaming transcription
- Real-time microphone recording
- Persistent storage
- User accounts
- Assessment history
- Progress tracking & waveform visualization